

符号计算入门

作者： R. De Graeve, B. Parisse, B. Ycart

机构： 格勒诺布尔约瑟夫·傅里叶大学 (Universit  Joseph Fourier,
Grenoble)

译者： Zongwen Chi (结合gemini 3.6 flash扩展模式)

Xcas 是一款开源的计算机代数系统 (CAS/符号计算软件)。

下载地址为： http://www-fourier.ujf-grenoble.fr/~parisse/giac_fr.html

它是与 Maple 和 MuPAD 相当的软件，并且与它们高度兼容。

可以对 Xcas 进行配置，使其能够接受 Maple、MuPAD 或 TI 计算器

(TI-89、Voyage 200、Nspire CAS) 的语法。本文将仅限于介绍 Xcas 本身的语法。

本入门教程旨在帮助具备一定数学基础 (相当于高中理科或大学理工科一年级水平) 并具备基本计算机操作经验的用户快速上手 Xcas。

本文并不会展示 Xcas 的所有功能。特别地，我们不会涉及交互式几何、Logo 海龟绘图以及电子表格功能。如需进一步的高级操作，请参考软件主页提供的在线帮助以及各类相关文档。

接下来的内容旨在通过介绍一些最常用的命令来帮助初学者。

建议在启动 Xcas 后阅读本文 (

在 Windows 下，点击快捷方式 xcasfr.bat；

在 Linux Gnome 下，点击 Education 菜单中的 Xcas，或在 Terminal 中输入 xcas & 并按下 Enter 键；

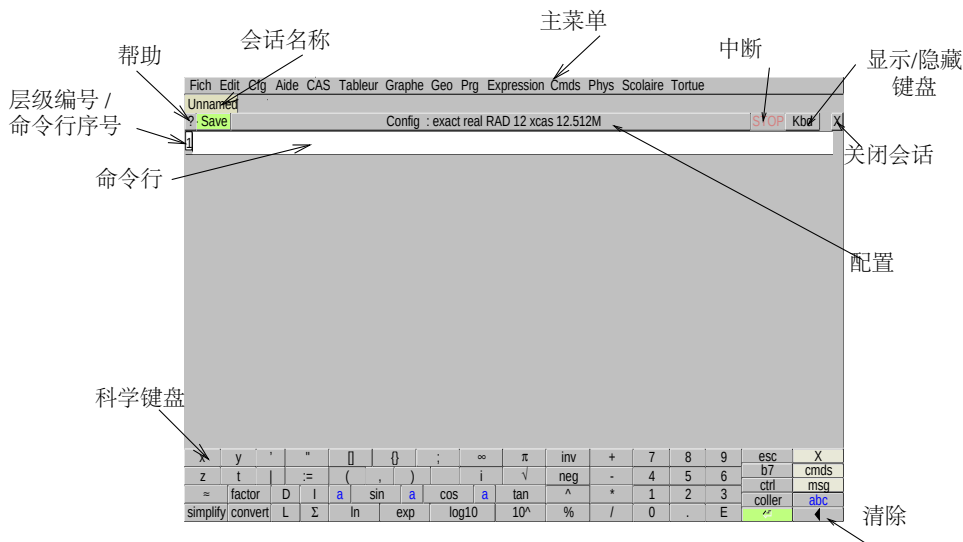
在 Mac 下，点击 Applications 菜单中的 Xcas)，并逐一执行文中给出的示例命令，以理解其作用与效果。

目录与索引存放在本文末尾，参见附录 A。

1 新手入门

1.1 界面

目前，您只需输入您的第一批命令。界面还提供了许多其他功能，您将在后续内容中逐步了解。启动 Xcas 时，界面如下图所示：



您可以调整其大小。自上而下，该界面依次包含：

- 一个可点击的菜单栏：Fich, Edit, Cfg, Aide, CAS, Expression, Cmd, Prg, Graphic, Geo, Tableur, 等。
- 一个显示会话名称的标签页，若会话尚未保存则显示为 Unnamed（可以并行打开多个会话，从而拥有代表各个会话的多个标签页），
- 一个会话管理区域，包含：
 - 一个用于打开帮助索引的 ? 按钮，
 - 一个用于保存会话的 Save 按钮，
 - 一个显示 CAS 配置 (Config: exact real ...) 并允许对其进行修改的按钮，
 - 一个红色的 STOP 按钮，用于中断耗时过长的计算，
 - 一个 Kbd 按钮，用于显示类似于计算器的虚拟键盘（如上图所示）。它可以方便您的输入，并可通过 Kbd->msg (或通过菜单 Cfg->Montrer->msg) 显示消息窗口，以及通过 Kbd->cmds (或通过菜单 Cfg->Montrer->bandeau) 显示命令面板，
 - 一个 X 按钮，用于关闭当前会话，
- 一个编号为 1 的白色矩形区域（第一条命令行），您可以在其中输入您的第一条命令（如有必要，请点击该命令行使光标出现）： $1+1$ ，随后按下“回车”键（依据键盘不同，标注为 Enter 或 Return）。计算结果将显示在下方，同时会自动打开一条编号为 2 的新命令行。您可以保存所作的修改。

您可以修改界面的外观，并保存您的修改以供日后使用（菜单 Cfg）。

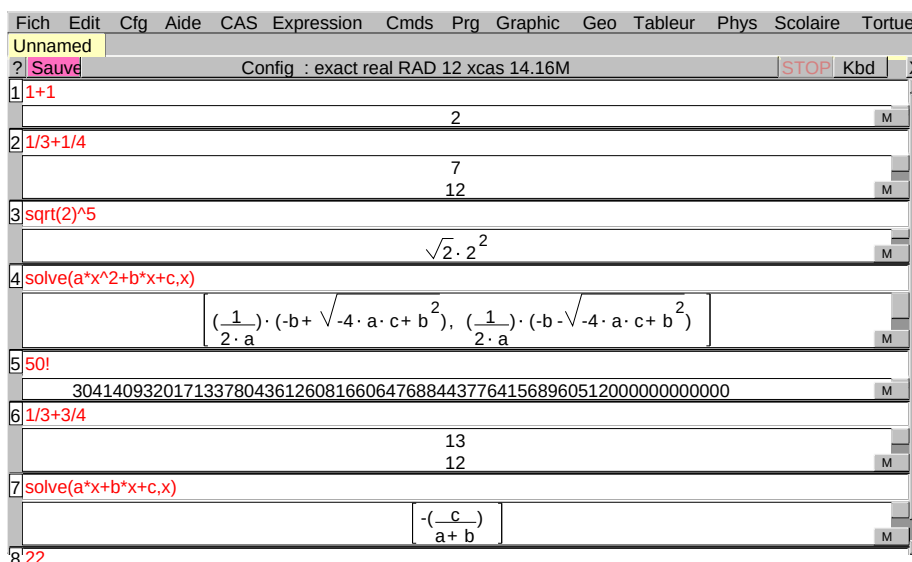
目前，您只需在接下来的各个命令中依次输入命令。如果您使用的是本教程的 HTML 版本，可以直接从浏览器中复制并粘贴文中给出的命令。每输入一条命令行，按下“回车”键 (Enter) 即可执行。例如，您可以尝试执行以下命令：

```
1/3+1/4
sqrt(2)^5
solve(a*x^2+b*x+c,x)
50!
```

所有命令都会保存在内存中。因此，您可以使用 Ctrl+↑ 组合键翻阅会话的历史记录，重新调出之前的命令并进行修改。例如，您可以尝试修改前面的命令并执行以下操作：

```
1/3+3/4
solve(a*x+b*x+c,x)
```

于是得到：



此时，右侧会出现一个滚动条，用于在会话的各个层级之间浏览，例如在此处可以查看第 8 个层级。 **Edit**（编辑）菜单允许您构建比单纯的命令序列更复杂的会话。您可以创建命令行分组（章节）、添加注释，或者将多个层级合并为一个层级。

1.2 命令与在线帮助

命令按主题分类列于顶部栏的菜单中: **CAS, Expression, Cmds, Prg, Graphic, Geo, Tableur, Phys, Scolaire, Tortue**. 某些菜单被称为“助手”（Assistant）菜单，这是因为其中命令按主题进行了分类并附有详细说明（如 **CAS** 菜单），或者因为在选择命令后，会弹出一个对话框要求您具体指定所选命令的参数（如 **Tableur** ► **Maths** 菜单或 **Graphic** 菜单）。

其他菜单包含了各种命令的名称： **Cmds** 菜单包含所有符号计算（calcul formel）命令， **Prg** 菜单包含编程中使用的所有命令， **Geo** 菜单包含所有几何命令……当您从菜单中选择一条命令时，

- 要么弹出一个对话框，允许您指定命令的参数（例如通过 **Graphic** 菜单绘制曲线，或通过 **Tableur** ► **Maths** 菜单进行统计分析）；
- 要么将该命令直接复制到命令行中。要了解该命令的语法，可以点击左上角的 ? 按钮，或显示消息区域（通过菜单 **Cfg** -> **Montrer** -> **msg**）。您还可以：
 - 打开所选命令的帮助索引（如果在通用配置菜单 **Cfg** -> **Configuration generale** 中勾选了 **Aide index automatique** 复选框，该操作会自动执行）。此时需点击 **OK** 按钮将命令复制到命令行中（前提是光标位于某条命令行内）。您也可以点击 **Details** 按钮，在浏览器中查看该命令对应的详细手册页面。
 - 在通用配置菜单（**Cfg** -> **Configuration generale**）中勾选 **Aide HTML automatique** 复选框，在浏览器中自动打开手册的对应页面。

Aide（帮助）菜单包含各种可能的帮助形式：用户指南（界面）、参考指南（**Manuels** -> **Calcul formel**，提供每条命令的详细帮助）、**Index**（按字母顺序排列的命令列表，带有方便快捷定位的输入框）以及关键词搜索。

如果您已经知道某条命令的名称并希望检查其语法（例如 **factor**），您可以输入命令名称的开头（例如 **fact**），然后按下 **Tab** 键（在法语键盘上位于 **A** 键左侧）或点击左上角的 ? 按钮。

此时，命令索引窗口将会弹出，自动定位到第一个可能的补全项，并显示每条命令的简要帮助信息。例如，如果您想因式分解一个多项式，猜想命令名称以 **fact** 开头，只需输入 **fact** 并按下 **Tab** 键，用鼠标选中 **factor**（或其中的一个示例），最后点击 **OK** 即可。

您也可以输入 **?factor** 来直接获取简要帮助。如果您输入的名称无法被识别，系统会为您推荐相近的命令。

1.3 输入命令

按“回车”键 (Enter) 即可直接执行一行命令。如果不希望显示运行结果，可以在命令行末尾加上 `;;`，然后再按“回车”键确认。也可以在执行前连续编写多条命令，只要用分号 (`;`) 将它们隔开即可。按“回车”键 (Enter) 即可直接执行一行命令。如果不希望显示运行结果，可以在命令行末尾加上 `;;`，然后再按“回车”键确认。也可以在执行前连续编写多条命令，只要用分号 (`;`) 将它们隔开即可。

刚开始使用时，许多错误都源于对数学表达式的格式转换不当：`Xcas` 无法直接理解您按日常习惯写出的数学式。虽然您可以通过键盘输入 $ax^2 + bx + c$ ，但计算机无法自动理解您是想对 x 求平方、将其与 a 相乘等操作。您必须明确指定每一个运算符，正确的语法应为 $a*x^2+b*x+c$ 。在输入命令时，乘法必须用星号 (`*`) 表示，而在系统返回的答案中，乘法则会显示为一个点号 (`.`)。我们需要特别强调的是：对于 `Xcas` 而言， ax 是一个由两个字母构成的单一变量名，而非 a 与 x 的乘积。

运算	
<code>+</code>	加法
<code>-</code>	减法
<code>*</code>	乘法
<code>/</code>	除法
<code>^</code>	乘方 / 幂运算

只要注意若干细节，表达式的书写就相当直观。圆括号的作用与通常一致，用于指定运算顺序；方括号则专门用于列表和索引（下标）。运算符的优先级遵循标准规则（乘法优先于加法，乘方优先于乘法）。例如：

- $2*a+b$ 会返回 $2 \cdot a + b$
- $a/2*b$ 会返回 $\frac{a \cdot b}{2}$
- $a/2/b$ 会返回 $\frac{a}{2 \cdot b}$
- $\text{normal}(a/2/b)$ 会返回 $\frac{a}{2 \cdot b}$
- a^2*b 会返回 $a^2 \cdot b$

如有疑问，加上括号总是更稳妥的做法，以确保运算顺序符合预期。

命令和对应的结果都是带有编号的。不过，如果您修改了某一条命令行，它仍会保留原有的编号。您可以通过 `ans()`（即 `answer`）来调取上一条结果，也就是最后一次执行/求值的命令所返回的答案。

2 符号计算的对象

2.1 数值

数分为精确数和近似数。精确数包括预定义常数、整数、整数分数（有理数），以及更广义地，任何仅包含整数和常数的表达式，例如 $\text{sqrt}(2)*e^{(i*\pi/3)}$ 。近似数采用标准的科学记数法表示：整数部分、小数点和小数部分（末尾可接字母 e 及指数）。例如， 2 是一个精确整数， 2.0 是该整数的近似版本； $1/2$ 是一个有理数， 0.5 是该有理数的近似版本。`Xcas` 可以处理任意精度的整数：您可以尝试输入 $500!$ ，数一数返回结果共有多少位数字。

通过 `evalf` 命令可以将精确值转换为近似值；通过 `exact` 命令可以将近似值转换为精确的有理数。如果参与计算的所有数字都是精确数，计算将在精确模式下进行；只要其中有一个数字是近似数，计算就会在近似模式下进行。因此， $1.5+1$ 会返回一个近似数，而 $3/2+1$ 则会返回一个精确数。

通过 `evalf` 命令可以将精确值转换为近似值；通过 `exact` 命令可以将近似值转换为精确的有理数。如果参与计算的所有数字都是精确数，计算将在精确模式下进行；只要其中有一个数字是近似数，计算就会在近似模式下进行。因此，`1.5+1` 会返回一个近似数，而 `3/2+1` 则会返回一个精确数。

```
sqrt(2)                                译注：根号2的精确值
evalf(sqrt(2))                        译注：根号2的近似值
sqrt(2)-evalf(sqrt(2))
exact(evalf(sqrt(2)))*10^9            译注：先计算根号2的浮点近似值，再将其转回精确的分数形式，最后放大 10^9倍
exact(evalf(sqrt(2)*10^9))            (即乘以10亿)。
```

对于近似实数，默认精度接近 14 位有效数字（根据 `Xcas` 版本的不同，规格化浮点实数的相对精度为 53 或 45 比特）。您可以通过将所需的小数位数作为 `evalf` 的第二个参数来提高精度。

```
evalf(sqrt(2),50)
evalf(pi,100)
```

此外，您也可以通过修改变量 `Digits` 来更改所有计算的默认精度。

```
Digits:=50
evalf(pi)
evalf(exp(pi*sqrt(163)))
```

字母 ``i`` 专门保留用于表示虚单位（根号-1），不能被重新赋值；特别是，不能将其用作循环索引（循环变量）。

```
(1+2*i)^2
(1+2*i)/(1-2*i)
e^(i*pi/3)
```

`Xcas` 区分无符号无穷大 `infinity` (∞)，正无穷大 `+infinity` 或 `inf(+∞)`，以及负无穷大 `-infinity` 或 `-inf(-∞)`。

```
1/0; (1/0)^2; -(1/0)^2
```

预定义常数	
<code>pi</code>	$\pi \simeq 3.14159265359$
<code>e</code>	$e \simeq 2.71828182846$
<code>i</code>	$i = \sqrt{-1}$
<code>infinity</code>	∞
<code>+infinity</code> ou <code>inf</code>	$+\infty$
<code>-infinity</code> ou <code>-inf</code>	$-\infty$

2.2 字符与字符串

字符串用双引号 (") 括起来。字符则是仅包含一个元素的字符串。

```
s:="azertyuiop"
size(s)
s[0]+s[3]+s[size(s)-1]
concat(s[0],concat(s[3],s[size(s)-1]))
head(s)
tail(s)
mid(s,3,2)
l:=asc(s)
ss:=char(l)
string(123)
expr(123)
expr(0123)
```

字符串	
asc	字符串->ASCII 码列表
char	ASCII码列表->字符串
size	字符数 / 字符串长度
concat 或 +	字符串拼接
mid	截取子字符串 / 提取片段
head	首字符
tail	去除首字符后的剩余字符串
string	数字或表达式->字符串
expr	字符串->数字（十进制或八进制）或表达式

2.3 变量

如果一个变量未包含任何值，我们就称其为形式变量（符号变量）：在被赋值之前，所有变量默认都是形式变量。赋值操作用 `:=` 表示。在会话开始时，`a` 是一个形式变量；但在执行指令 `a:=3` 之后，`a` 就变成了已赋值变量，随后的所有计算中 `a` 都会被替换为 `3`，输入 `a+1` 将返回 `4`。会保存您当前会话中的所有内容。如果希望变量 `a` 在赋值后重新变回形式变量，必须通过 `purge(a)` 将其“清空/清除”

切勿混淆：

- 符号 `:=` 表示赋值操作
- 符号 `==` 表示布尔/逻辑相等（这是一个二元运算符，返回 `1` 或 `0`；其中 `1` 代表 `true` 即“真”，`0` 代表 `false` 即“假”）。
- 符号 `=` 用于定义方程

```

a==b
a:=b
a==b
solve(a=b, a)
solve(2*a=b+1, a)

```

可以使用 `assume` 命令对变量添加某种假设，例如 `assume(a>2)`。假设是一种特殊形式的赋值，它会清除该变量此前可能已被赋予的值。在计算过程中，变量不会被直接替换为数值，但系统会尽可能地利用该假设。例如，在设置了假设 `a>2` 后，输入 `abs(a)` 将会直接返回 `a`。

```

sqrt(a^2)
assume(a<0)
sqrt(a^2)
assume(n, integer)
sin(n*pi)

```

`subst` 函数允许在不影响变量本身（即不对其进行赋值）的前提下，将表达式中的某个变量替换为一个数值或另一个表达式。

```

subst(a^2+1, a=1)
subst(a^2+1, a=sqrt(b-1))
a^2+1

```

注意：若要通过引用方式将值存入变量（例如修改列表、向量或矩阵中的某个元素），并在不重新创建新列表的前提下原地修改（**in-place**）现有列表，应使用指令 `=<` 来代替 `:=`。该指令比 `:=` 更高效，因为它省去了复制整个列表的时间开销。

2.4 表达式

表达式是由数字和变量通过运算符组合而成的结构，例如 `x^2+2*x+c`。

当确认/执行一条命令时，`Xcas` 会将变量替换为其对应的值（如果该变量已被赋值），并执行相应的计算操作。

```

(a-2)*x^2+a*x+1
a:=2
(a-2)*x^2+a*x+1

```

在求值（计算）过程中，某些化简操作会自动执行：

- 整数与分数的运算，包括将其约分为最简分数（既约分数）形式。
- 平凡的化简（基础化简），例如 $x+0 = x$ 、 $x-x = 0$ 、 $x*1 = x$ 等。
- 一些三角函数化简，例如 $\cos(-x) = \cos(x)$ 、 $\cos(\pi/4) = (1/2)^{(1/2)}$ 、 $\tan(\pi/4) = 1$ 等。

我们将在下一节中了解如何进行更多的化简。

2.5 展开与化简

除上一节所述的规则外，系统不会进行自动的化简。这主要有两个原因：
计算开销：非平凡（复杂）的化简有时非常耗时，因此是否执行化简完全由用户决定；多解性：根据具体的使用目的，同一个表达式通常存在多种不同的化简形式。用于转换表达式的主要命令如下：

- **expand**：展开表达式，仅考虑乘法对加法的分配律以及整数幂的展开。
- **normal** 与 **ratnormal**：在执行效率与化简效果之间具备良好的平衡，它们将有理分式（两个多项式的比值）写成已展开的最简分式形式；**normal** 会考虑代数数（例如 $\sqrt{2}$ ），而 **ratnormal** 则不会。两者均不考虑超越函数之间的关系（例如 $\sin$ 与 $\cos$ ）。
- **factor**：比前面的函数稍慢，它将分式写成因式分解后的最简形式。
- **simplify**：尝试在应用 **normal** 之前先将表达式归结为代数独立的变量。这种方法时间开销更大且过程“不可见”（无法控制中间的重写步骤）。涉及代数扩张（例如平方根）的化简，有时需要调用两次该函数，和/或配合假设（**assume**）消除绝对值后，才能得到理想的化简结果。
- **tsimplify**：尝试归结为代数独立的变量，但随后不会应用 **normal**。

在顶部菜单栏的 **Expression**（表达式）菜单中，各个子菜单均属于重写（改写）菜单，其中包含了更多用于进行特定或专用转换的函数。

```
b:=sqrt(1-a^2)/sqrt(1-a)
ratnormal(b)
normal(b)
tsimplify(b)
simplify(b)
simplify(simplify(b))
assume(a<1)
simplify(b)
simplify(simplify(b))
```

convert 函数允许将一个表达式转换为另一个等价的形式，具体的转换格式由第二个参数指定。

```
convert(exp(i*x), sincos)
convert(1/(x^4-1), partfrac)
convert(series(sin(x), x=0, 6), polynom)
```


变换	
simplify	化简:深度化简
tsimplify	化简 (较弱版本): 较轻量的化简
normal	标准型 / 规范形式: 化简为展开的最简分式, 支持代数数
ratnormal	标准型 (较弱版本): 针对纯有理式的规范化
expand	展开: 展开乘法与整数幂
factor	因式分解: 对表达式进行因式分解
assume	添加假设: 为变量设定条件约束
convert	转换为指定格式: 按指定格式重写表达式

2.6 函数

Xcas中预定义了许多函数, 特别是各种常见的初等/经典函数。最常用的函数如下表所示; 对于其他函数, 可参见菜单 **Cmds->Reel->Special**。

此外, 用户也可以自定义函数, 例如:

– 单变量函数的定义:

```
f(x) :=x*exp(x) ou
f :=x->x*exp(x) ou
f :=unapply(x*exp(x),x)
```

– 导函数的定义: :

```
g :=function_diff(f) ou
g :=unapply(diff(f(x),x),x))
```

注意 $g(x) :=diff(f(x),x)$ 是无效的! 因为 `:=` 右侧的表达式在定义函数时不会被立即求值/评估.....这种情况下必须使用 `unapply`。

– 二元函数的定义:

```
h(r,t) :=(r*exp(t),r*t) 或
h :=(r,t)->(r*exp(t),r*t);
```

– 由二元函数构造一个“将单变量映射为另一个函数”的函数:

```
k(t) :=unapply(h(r,t),r)
```

注意: 这里的 $k(t)$ 本身是一个关于变量 r 的函数, 它将 r 映射为 $h(r,t)$ 。例如: $k(1)(2)=(2*exp(1),2)$ 。在这种情况下, 同样必须使用 `unapply`。

常见函数	
abs	绝对值
sign	符号函数 (-1,0,+1)
max	最大值
min	最小值
round	四舍五入 / 舍入
floor	向下取整 / 整数部分(<= 该数值的最大整数)
frac	小数部分
ceil	向上取整(>=该数值的最小整数)
re	实部
im	虚部
abs	模 / 复数模长
arg	幅角
conj	共轭复数
affixe	复数坐标 / 复点
coordonees	坐标
factorial 或 !	阶乘
sqrt	平方根
exp	指数函数
log	自然对数
ln	自然对数
log10	常用对数 / 以 10 为底的对数
sin	正弦
cos	余弦
tan	正切
cot	余切
asin	反正弦
acos	反余弦
atan	反正切
sinh	双曲正弦
cosh	双曲余弦
tanh	双曲正切
asinh	反双曲正弦
acosh	反双曲余弦
atanh	反双曲正切

要创建一个新函数，需要借助包含该变量的表达式来进行声明。例如，表达式 $x^2 - 1$ 的定义形式为 $x^2 - 1$ 。要将其转换为将 x 映射为 $x^2 - 1$ 的函数 f ，存在以下三种方法：

```
f(x) := x^2 - 1
f := x -> x^2 - 1
f := unapply(x^2 - 1, x)
```

```
f(2);
f(a^2);
```

如果 f 是单变量函数，且 E 是一个表达式，则 $f(E)$ 是另一个表达式。切记不可混淆函数与表达式。

如果定义 $E := x^2 - 1$ ，那么变量 E 中保存的就是表达式 $x^2 - 1$ 。若要获取该表达式在 $x = 2$ 时的值，必须写作 $\text{subst}(E, x=2)$ ，而不能写作 $E(2)$ ，因为 E 并不是一个函数。

在定义函数时，赋值符号右侧的内容不会被立即求值/评估。因此写作

```
E:=x^2-1;
```

$f(x) := E$ 定义的实际上是函数 $f: x \mapsto E$ （因为 E 本身未被展开求值）；

相比之下，写作 $E := x^2 - 1$ ； $f := \text{unapply}(E, x)$ 则能正确定义函数 $f: x \mapsto x^2 - 1$ （因为 E 在此时被求值并替换为了对应的表达式）。

函数之间可以进行加法或乘法运算，例如： $f := \sin * \exp$ 。要对函数进行复合，可以使用 $@$ 运算符；若要将一个函数与自身连续复合多次，则使用 $@@$ 运算符。

```
f:=x->x^2-1;
(f@f)(2);
(f@sqrt)(a);
f1:=f@sin
f2:=f@f
f3:=f@@3
f1(a)
f2(a)
f3(a)
```

可以定义取值为 $\mathbb{R}$ 的多元函数，例如：

```
f(x,y):=x+2*y
```

以及取值为 $\mathbb{R}^p$ 的多元函数，例如：

```
f(x,y):=(x+2*y,x-y)
```

2.7 列表、序列与集合

`xcas` 区分了多种由逗号分隔的对象集合类型：

- 列表（位于方括号内）
- 序列（位于圆括号内）
- 集合（位于百分号-大括号内，如 `%{ % }`）

```
liste:=[1,2,4,2]
sequence:=(1,2,4,2)
ensemble:=%{1,2,4,2%}
```

列表可以嵌套列表（矩阵即属于此类），而序列则是扁平的（序列中的元素不能再是序列）。在集合中，元素的顺序并不重要，且每个元素都是唯一的。此外还存在另一种称为表（`table`）的结构，我们将在后文中另作介绍。

只需将序列括在方括号内即可将其转换为列表，或括在带 `%` 前缀的大括号内即可将其转换为集合。使用 `op` 可以将列表转换为其对应的序列；而将序列括在方括号内或使用 `nop` 函数，则将其转换为对应的列表。列表的元素数量可以通过 `size`（或 `nops`）函数获取。

```

se:=(1,2,4,2)
li:=[se]
op(li)
nop(se)
nops(se)
%{se%}
size([se])
size(%{se%})

```

要创建列表或序列，可以使用迭代命令，例如 **\$** 或 **seq**（对表达式进行迭代），或者使用 **makelist**（借助函数来定义列表）。

```

1$5
k^2 $ (k=-2..2)
seq(k^2,k=-2..2)
seq(k^2,k,-2..2)
[k^2$(k=-2..2)]
seq(k^2,k,-2,2)
seq(k^2,k,-2,2,2)
makelist(x->x^2,-2,2)
seq(k^2,k,-2,2,2)
makelist(x->x^2,-2,2,2)

```

空序列记为 **NULL**，空列表记为 **[]**。添加元素：要向序列中添加元素，只需用逗号将序列与新元素分隔开即可；要向列表中添加元素，则使用 **append** 函数。

元素访问：可以通过方括号内的索引来访问列表或序列中的元素，其中第一个元素的索引为 0。

```

se:=NULL; se:=se,k^2$(k=-2..2); se:=se,1
li:=[1,2]; (li:=append(li,k^2))$(k=-2..2)
li[0], li[1], li[2]

```

多项式通常由表达式定义，但也可以表示为其系数按降幂（次数降序）排列的列表，界定符为 **poly1[** 和 **]**。同时还存在多元多项式的表示方法。函数 **symb2poly** 与 **poly2symb** 于在“表达式表示法”与“列表表示法”之间进行相互转换。其第二个参数用于指定处理的是一元多项式（传入变量名）还是多元多项式（传入变量列表）。

序列与列表	
<code>E\$(k=n..m)</code>	创建序列
<code>seq(E, k=n..m)</code>	创建序列
<code>[E\$(k=n..m)]</code>	创建列表
<code>makelist(f, k, n, m, p)</code>	创建列表
<code>op(li)</code>	从列表转换为序列
<code>nop(se)</code>	从序列转换为列表
<code>nops(li)</code>	元素数量 / 元素个数
<code>size(li)</code>	元素数量 / 元素个数
<code>sum</code>	元素求和
<code>product</code>	元素求积
<code>cumSum</code>	累加和
<code>apply(f, li)</code>	对列表元素应用函数
<code>map(li, f)</code>	对列表元素应用函数
<code>map(li, f, matrix)</code>	对矩阵元素应用函数
<code>poly2symb</code>	由列表构造多项式
<code>symb2poly</code>	提取多项式系数

2.8 计算时间与内存占用

符号计算（计算机代数）面临的主要问题是中间计算的复杂性。这种复杂性既体现在执行命令所需的时间上，也体现在所需的内存空间上。**xcas** 函数中内置的算法非常高效，但它们无法保证在所有情况下都是最优解。

获取执行时间：函数 **time** 可以用来查看某条命令的执行时间（如果执行时间极短，**xcas** 会多次重复运行该命令以返回更精确的时间测量）。

查看内存使用：在 **Unix** 版本中，所使用的内存大小会显示在状态栏（按钮栏）中。

如果某条命令的运行时间超过了数秒，很有可能是发生了输入错误。请毫不犹豫地中断执行（点击右上角红色的 **STOP** 按钮；在此之前，建议先对当前会话进行保存）。

3 分析工具

3.1 导数

函数 **diff** 用于计算表达式关于其一个或多个变量的导数。

要对函数 f 求导，可以对表达式 $f(x)$ 调用 **diff**，但这样得到的结果是一个表达式。如果希望直接定义导函数，则需要使用 **function_diff**。

```
E:=x^2-1; diff(E);
f:=unapply(E,x); diff(f(x));
f1:=function_diff(f);f1(x);
```

不应该通过 `f1(x):=diff(f(x))` 的方式来定义导函数，因为在这样的定义中，`x` 会同时具有两个互不兼容的含义：一方面它是求导的形式变量，另一方面它又是函数 `f1` 的自变量。

另一方面，这种定义方式会在每次调用函数时都重新对 `diff` 进行一次求值（因为赋值语句右侧的内容默认不会被立即求值），这会导致运行效率非常低下。

应当使用 `f1:=function_diff(f)`，或者

`f1:=unapply(diff(f(x)),x)`。

函数 `diff` 适用于任意变量的组合，能够用于计算连续的偏导数（高阶偏导数）。

```
E:=sin(x*y)
diff(E,x)
diff(E,y)
diff(E,x,y)-diff(E,y,x)
simplify(ans())
diff(E,x$2,y$3)
```

如果 `diff` 的第二个参数是一个列表，则会返回一个由各导数组成的列表。例如，要计算 `sin(xy)` 的梯度：

```
diff(sin(x*y),[x,y]) //计算 sin(x*y) 关于 x 和 y 的偏导数列表（即梯度）
```

（也可以直接使用 `grad` 命令）。

此外，还有一些特定命令可用于计算偏导数的经典（算符）组合。

导数	
<code>diff(ex,t)</code>	表达式关于 t 的导数
<code>function_diff(f)</code>	函数的导函数
<code>diff(ex,x\$n,y\$m)</code>	偏导数
<code>grad</code>	梯度
<code>divergence</code>	散度
<code>curl</code>	旋度
<code>laplacian</code>	拉普拉斯算子
<code>hessian</code>	黑塞矩阵

3.2 极限与泰勒展开

函数 `limit` 用于计算有限极限或无穷极限（在极限存在的情况下）。
单侧极限：可以通过传入第四个参数（+1 或 -1）来求解右极限或左极限。

参数假设：当函数依赖于某个参数时，计算出的极限结果可能取决于使用 `assume` 函数对该参数所设定的假设条件。

```
limit(1/x,x,0)
limit(1/x,x,0,1)
limit(1/x,x,0,-1)
limit(a/x,x,0,1)
assume(a>0)
limit(a/x,x,0,1)
```

对于泰勒展开，有两个函数可用：`series` 和 `taylor`。

两者的区别在于：使用 `series` 时必须显式指定展开的阶数，而 `taylor` 的默认展开阶数为 6。

用户请求的泰勒展开阶数由 **Xcas** 在内部用于进行展开运算。若计算过程中发生了抵消与化简，最终获得的展开阶数可能会低于预期，此时需要传入更大的阶数重新进行计算。

返回的表达式由泰勒多项式和余项组成。余项中会出现一个 **order_size** 函数，该函数满足：对任意 $a > 0$ ，当 $x \rightarrow 0$ 时， $x^a \text{order_size}(x) \rightarrow 0$ 。若要去除余项并仅保留泰勒多项式，可以使用带有 **polynom** 选项的 **convert** 函数。

```
taylor(1/(x^2+1),0)
taylor(1/(x^2+a^2),x=0)
series(1/(x^2+1),0,11)
series(1/(x^2+1),+infinity,11)
series(tan(x),pi/4,3)
series(sin(x)^3/((1-cos(x))*tan(x)),0,4)
series(sin(x)^3/((1-cos(x))*tan(x)),0,6)
series(tan(sin(x))-sin(tan(x)),0,13)
convert(ans(),polynom)
series(f(x),0,3)
g:=f@f; series(g(x),0,2)
```

极限与泰勒展开	
<code>limit(ex,x,a)</code>	在a处的极限
<code>limit(ex,x,a,1)</code>	在a处的右极限
<code>limit(ex,x,a,-1)</code>	在a处的左极限
<code>taylor(ex,a)</code>	在a处的6阶泰勒展开
<code>series(ex,a,n)</code>	在a处的n阶泰勒展开

3.3 原函数与积分

函数 **int** 用于计算表达式关于 x 或指定变量的原函数（不定积分）。

指定积分变量：如果表达式中包含多个变量，需要明确指定对哪一个变量进行积分。

定积分计算：在积分变量之后追加两个参数 a 和 b ，即可计算在区间 $[a,b]$ 上的定积分。

```
int(x^2-1)
int(x^2-1,x,-1,1)
int(x*y,x)
int(x*y,y,0,x)
int(int(x*y,y,0,x),x,0,1)
```

为了计算积分，符号计算软件（计算机代数系统）会先寻找原函数，随后在积分上下限之间对其求值，以获得精确值。

但在某些情况下，寻找原函数是没有必要的：要么是因为不存在能够用初等函数表示的原函数，要么是因为数值计算更加适用（例如

原函数的计算时间过长、函数在积分区间内存在奇点等)。

在这种情况下，可以通过使用`evalf`来求解近似值（数值解），或者直接使用由 `evalf` 调用的 `romberg` 函数。

```
int(exp(-x^2))
int(exp(-x^2),x,0,10)
evalf(int(exp(-x^2),x,0,10))
romberg(exp(-x^2),x,0,10)
ans()/sqrt(pi))
```

积分	
<code>int(E)</code>	表达式的原函数
<code>int(E,x,a,b)</code>	精确积分
<code>romberg(E,x,a,b)</code>	近似积分 / 数值积分

3.4 方程求解

与积分计算类似，方程求解也分为以下两种方式：

- 精确求解：在可能的情况下返回方程的所有精确解（例如对于某些多项式方程，或可以化归为多项式方程的情形）。
- 近似求解：通过迭代算法计算出其中某一个解的数值近似值。

精确求解通过 `solve` 函数实现，其第一个参数为一个方程。如果未显式指定方程的右端项，系统默认右端项为0。默认情况下，`solve` 不会返回复数解。若要获取复数解，需要在 CAS 配置中勾选 **Complex**（操作路径: `Cfg -> CAS Configuration`，或点击状态栏/按钮栏上的 `Config : exact...`）。

请在激活 `Complex` 选项之前和之后，分别执行以下命令：

```
solve(x^2-a*x+2,x)
solve(x^2+2,x)
solve(x^3=1,x)
```

对于1次和2次多项式，系统可以计算出精确根（针对3次和4次多项式的卡尔丹公式（`Cardan`）和法拉利公式（`Ferrari`）默认不被使用，因为求解出的表达形式极其繁琐、难以处理）。对于更高次数的多项式，`solve` 函数会显示一条错误信息并返回一个空列表。

对于三角方程，系统默认仅返回主解（主要区间内的解）。若要获取所有解（通解），需要启用`All_trig_sol`选项。

请对比在启用与未启用该选项时执行以下命令的效果：

```
solve(cos(x),x)
solve(cos(x)+sin(x),x)
```

函数也可以用来求解方程组。其中，第一个参数是方程列表，第二个参数是变量列表。


```
solve([x^2+y-2,x+y^2-2],[x,y])
```

近似求解函数为 **fsolve**。它在菜单中提供了多种可选算法（菜单路径为 **Calc->Num_solve_eq** 以及 **Calc->Num_solve_syst**）。

其中最著名的是牛顿法（**Algorithme de Newton**），该算法有多种变体。所有这些数值算法的基本原理，都是通过计算一个迭代序列的连续项，使其逐步收敛到方程或方程组的解。为此，通常需要根据具体情况指定一个初始点（起点）或一个搜索区间。

```
fsolve((x^5+2*x+1)=0,x,1,newton_solver)
newton(x^5+2*x+1,x,1.0)
newton(x^5+2*x+1,x,1+i)
newton(x^5+2*x+1,x,-1+i)
```

方程	
<code>solve(eq,x)</code>	方程的精确求解
<code>solve([eq1,eq2],[x,y])</code>	方程组的精确求解
<code>fsolve(eq,x)</code>	方程的近似求解
<code>fsolve([eq1,eq2],[x,y])</code>	方程组的近似求解
<code>newton</code>	牛顿法
<code>linsolve</code>	线性方程组
<code>proot</code>	多项式的近似根

3.5 常微分方程

与前两节类似，微分方程的求解也分为精确求解（并非所有方程都能求出解析解）和数值近似求解。

精确求解：通过 **desolve** 函数实现。

导数表示：未知函数 y 的导数可以直接写为 y' , y'' 等形式，系统内部会自动将其转化为 **diff(y)**, **diff(diff(y))**。

任意常数：如果未指定初始条件，计算结果将以任意常数的形式给出通解。

```
desolve(y'=y,y)
desolve(y''+2*y'+y=0,y)
desolve((x^2-1)*y'+2*y=0,y)
```

初始条件在 Xcas 中被视为额外的方程，它们与微分方程本身一同组成一个列表传递给 **desolve** 函数。

```
desolve([y'=y,y(0)=1],y)
desolve([y''+2*y'+y=0,y(0)=1],y)
desolve([y''+2*y'+y=0,y(0)=1,y'(0)=1],y)
desolve([y''+2*y'+y=0,y(0)=1,y(1)=0],y)
desolve([(x^2-1)*y'+2*y=0,y(0)=1],y)
desolve((t^2-1)*diff(y(t),t)+2*y(t)=0,y(t))
```

函数 **odesolve** 允许通过数值方法求解过点 (x_0, y_0) 的常微分方程 $y' = f(x, y)$
例如：

```
odesolve(sin(x*y), [x, y], [0, 1], 2)
```

用于计算 $y(2)$ ，其中 $y(x)$ 是满足初始条件 $y(0) = 1$ 的常微分方程 $y'(x) = \sin(xy)$ 的数值解。**plotode** 函数用于绘制微分方程解的图像，**plotfield** 用于绘制切线场（方向场/斜率场）。**interactive_odeplot** 函数则在绘制切线场的同时提供交互功能，允许在图表上点击任意位置，直接生成并绘制经过该点击点的积分曲线（解曲线）。

```
plotfield(sin(x*y), [x, y])
plotode(sin(x*y), [x, y], [0, 1])
erase()
interactive_plotode(sin(x*y), [x, y])
```

常微分方程	
desolve	精确求解 / 解析解
odesolve	近似求解 / 数值解
plotode	轨迹 / 解曲线绘制
plotfield	矢量场 / 切线场绘制
interactive_plotode	可点击交互界面

4 代数工具

4.1 整数算术

整数的相关运算存放在菜单 **Cmds->Entier** 中。模 p 算术（同余计算）可以通过 **% p** 来实现。一旦定义了一个模 p 的整数（例如 **a:=3%5**），后续所有涉及该变量的计算都将在同余类环 $\mathbb{Z}/p\mathbb{Z}$ 中自动完成：**a*2** 返回 **1%5**（因为 $6 \equiv 1 \pmod{5}$ ），**1/a** 返回 **2%5**，... 若要高效计算模 p 幂次（快速模幂运算），既可以直接使用模 p 整数求幂，也可以使用 **powermod** 或 **powmod**。

```
a:=3%5
a+12
a^4
powermod(3,4,5)
```

整数	
<code>a%p</code>	$a \bmod p$
<code>powmod(a,n,p)</code>	$a^n \bmod p$ (模幂运算)
<code>irem</code>	欧几里得除法的余数
<code>iquo</code>	欧几里得除法的商
<code>iquorem</code>	商和余数
<code>ifactor</code>	质因数分解
<code>ifactors</code>	质因数列表
<code>idivis</code>	约数列表
<code>gcd</code>	最大公约数
<code>lcm</code>	最小公倍数
<code>iegcd</code>	裴蜀等式
<code>iabcuv</code>	返回系数 $[u,v]$ 且满足 $au + bv = c$
<code>is_prime</code>	判断整数是否为质数
<code>nextprime</code>	下一个质数
<code>previousprime</code>	上一个质数

4.2 多项式与有理分式

多项式处理的相关函数存放在菜单 **Cmds** -> **Polyn\^omes**.

使用 **normal** 或 **expand** 进行展开，或者更常见地用于将分式化简为既约分式（最简分式）。使用 **factor** 进行因式分解。运算结果取决于所选取的数域：默认情况：如果系数是精确数，计算将在有理数域上进行；如果包含浮点数，则在实数域上进行。复数域：若需要在复数域（精确值或近似值）上计算，需要通过状态栏/配置按钮 **Config:...** 激活 **Complexe** 选项。

有限环与有限域：可以将系数声明为模 p 的整数，以便在有限环 $\mathbb{Z}/p\mathbb{Z}$ 中工作（使用 **%** 命令），或使用 **GF** 命令在有限域（Galois Field）中工作。请在激活 **Complexe** 选项之前和之后，分别执行以下命令：

```
P:=x^4-1
factor(P)
gcd(P,x^3-1)
divis(P)
propfrac(x^4/P)
partfrac(4/P)
Q:=(x^4+1)%3
factor(Q)
G:=GF(2,8,['a','G'])
factor(G(a^3)*x^2+1)
genpoly(5,3,x)
genpoly(2,3,x)
genpoly(2*y+5,3,x)
```